

TASK 05 — Redis Caching & Distributed Data Architecture

Step 1 — Understand caching

What is caching

Caching means temporarily storing frequently used data in a fast storage system so that we don't have to fetch the same data from the database every time.

Example:

Without cache:

Client



Spring Boot



Database



Product details

Every request goes to the database.

With Redis cache:

Client



Spring Boot



Redis Cache



If data exists → Return data



If data doesn't exist



Database → Store in Redis → Return data

Redis is commonly used because it is an **in-memory data store**, so reading data from it is usually much faster than querying a database.

1. Why is caching required

Suppose we have a product API:

GET /products/101

The database contains:

ID: 101

Name: Laptop

Price: ₹60,000

If 10,000 users request the same product, without caching:

10,000 requests

↓

10,000 database queries

This can increase:

- Database load
- Response time
- CPU usage
- Network traffic

With Redis:

First request

↓

Redis → MISS

↓

Database

↓

Store result in Redis

Next 9,999 requests



Redis → HIT



Return data

Therefore, caching improves **performance and reduces database load**.

2. Cache Hit

A **cache hit** occurs when the requested data is already available in the cache.

Example:

Request:

GET /products/101

Redis:

product:101 → FOUND

Result:

Return product from Redis

Flow:

Client



Spring Boot



Redis



Data found?



Return data

Simple example

Redis contains:

product:101 = Laptop, ₹60,000

User requests product 101.

Redis finds it.

That is a **Cache Hit**.

Advantage

The application doesn't need to query the database.

3. Cache Miss

A **cache miss** occurs when the requested data is **not available in Redis**.

Example:

GET /products/101

Redis checks:

product:101 → NOT FOUND

So the application queries the database:

Redis

↓

MISS

↓

Database

↓

Get product

↓

Store product in Redis

↓

Return product

Next time the same product is requested, Redis can return it directly.

Example

First request:

Client → Redis → MISS → Database → Redis → Client

Second request:

Client → Redis → HIT → Client

4. Cache Expiration

Cache data should not necessarily remain in Redis forever.

Cache expiration means automatically removing cached data after a specified period.

This period is called **TTL — Time To Live**.

For example:

product:101

TTL = 10 minutes

After 10 minutes:

product:101 → Expired

Then the next request causes a cache miss:

Client

↓

Redis

↓

Expired → MISS

↓

Database

↓

New data

↓

Redis

Why expiration is useful

Suppose the product price changes:

Old price:

₹60,000

But Redis still contains the old value.

Expiration ensures that cached data is periodically refreshed.

5. Cache Invalidation

Cache invalidation means deliberately removing or updating cached data when the original data changes.

For example:

Initially:

Database:

Product 101 = ₹60,000

Redis:

product:101 = ₹60,000

Now the price changes:

Database:

Product 101 = ₹55,000

But Redis may still contain:

product:101 = ₹60,000

If we return Redis data, users see the wrong price.

So we invalidate the cache:

Database updated

↓

Delete Redis entry

↓

product:101 removed

Next request:

Redis → MISS



Database → ₹55,000



Redis updated



Client gets ₹55,000

Difference between expiration and invalidation

Concept	Meaning
----------------	----------------

Expiration	Cache automatically disappears after TTL
-------------------	--

Invalidation	Application deliberately removes/updates cache
---------------------	--

Cache Hit	Data found in cache
------------------	---------------------

Cache Miss	Data not found in cache
-------------------	-------------------------

6. Cache-Aside Pattern

This is one of the most commonly used caching strategies.

The application is responsible for reading and writing the cache.

Read operation

Request



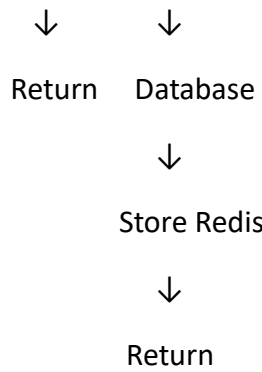
Spring Boot



Redis



HIT MISS



Step-by-step

Step 1

Application checks Redis.

GET product:101

Step 2 — If found

Return the data.

Redis → Product → Client

Step 3 — If not found

Query database.

Redis → MISS



Database

Step 4

Store the database result in Redis.

Database



Redis

Step 5

Return the result to the client.

Step 2 — Set Up Redis

we need to install and run Redis.

You can run Redis:

- Locally
- Docker
- AWS
- Redis Cloud
- Another cloud provider

For development, Docker is very convenient.

Example:

```
docker run -d \  
  --name redis \  
  -p 6379:6379 \  
  redis
```

Redis normally uses:

Host: localhost

Port: 6379

You can verify Redis using:

```
redis-cli ping
```

Expected result:

```
PONG
```

This confirms Redis is running.

Step 3 — Add Redis to Spring Boot

Now connect Spring Boot with Redis.

For Maven, add the Spring Data Redis dependency:

```
<dependency>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter-data-redis</artifactId>  
</dependency>
```

If you want to use Spring's caching annotations, also add:

```
<dependency>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter-cache</artifactId>  
</dependency>
```

Configure Redis

In: src/main/resources/application.properties

add:

spring.data.redis.host=localhost

spring.data.redis.port=6379

For some Spring Boot versions/configurations, the property names may be:

spring.redis.host=localhost

spring.redis.port=6379

Use the property names appropriate to your Spring Boot version.

Step 4 — Select Cacheable Data

1. Product details

Example:

Product ID: 101

Name: Laptop

Price: ₹60,000

Category: Electronics

If product information is requested frequently, caching can reduce database queries.

2. User profile

Example:

User ID: 1

Name: Ravi

Email: ravi@example.com

Frequently accessed profile information can be cached when appropriate.

3. Frequently accessed configuration

For example:

Application configuration

Country settings

Feature configuration

Category information

If the data changes rarely but is requested frequently, caching can be very useful.

Step 5 — Implement Cache Read

When a user calls an API, the application first checks **Redis** instead of directly querying the database.

For example:

GET /products/101

Spring Boot receives the request.

Step 1: API Request

Client



GET /products/101

Step 2: Check Redis

Spring Boot checks whether Redis contains:

product:101

Suppose Redis contains:

product:101

```
{  
  "id": 101,  
  "name": "Laptop",  
  "price": 60000  
}
```

Then Redis returns the product.

Step 3: Cache Hit

Because the data exists in Redis:

Cache Hit

The application immediately returns the data.

Client



Spring Boot



Redis



Cache HIT



Product



Client

Important point

The database is **not accessed** when there is a cache hit.

This makes the response faster and reduces database load.

Step 6 — Implement Cache Miss

This is what happens when Redis **does not contain the requested data**.

Example

User requests:

GET /products/200

Spring Boot checks Redis:

product:200

But Redis doesn't have it.

Therefore:

Cache Miss

Step-by-step

Step 1 — API Request

Client



GET /products/200

Step 2 — Check Redis

Spring Boot



Redis



product:200 ?

Redis responds:

NOT FOUND

This is a **cache miss**.

Step 3 — Go to Database

Spring Boot now queries the database:

```
SELECT * FROM products WHERE id = 200;
```

Database returns:

ID: 200

Name: Mobile Phone

Price: ₹25,000

Step 4 — Store result in Redis

Now Spring Boot stores the database result in Redis:

product:200

↓

```
{  
  "id": 200,  
  "name": "Mobile Phone",  
  "price": 25000  
}
```

Now Redis has the data for future requests.

Step 5 — Return response

Finally:

Database

↓

Spring Boot

↓

Client

The client receives:

```
{  
  "id": 200,  
  "name": "Mobile Phone",  
  "price": 25000  
}
```

Step 7 — Implement Cache Expiration

Configure an appropriate TTL

TTL means:

Time To Live

It defines **how long data should remain in Redis**.

For example:

product:200

TTL = 10 minutes

This means Redis keeps the cached product for 10 minutes.

After 10 minutes: product:200 → Expired

The next request will cause a cache miss.

Example Suppose:

10:00 AM → Product stored in Redis

TTL = 10 minutes

Then:

10:00 → Cached

10:05 → Available

10:09 → Available

10:10 → Expired

At 10:11:

API Request

↓

Redis

↓

MISS

↓

Database

The application gets the latest data and caches it again.

Why do we need TTL

Imagine a product price:

Database:

₹60,000

Redis also stores:

₹60,000

Later the database changes:

₹55,000

If Redis keeps the old value forever, users could receive:

₹60,000

instead of:

₹55,000

TTL helps prevent stale data from remaining in the cache indefinitely.

Cache Hit vs Cache Miss

Cache Hit

Data exists in Redis

No database query

Faster response

Reduces DB load

Return Redis data

Cache Miss

Data doesn't exist in Redis

Database query required

Slower than a hit

Increases DB load

Get DB data and cache it

Simple Spring Boot Example

With Spring Boot, you can implement this using Spring Cache.

1. Enable caching

```
@SpringBootApplication
```

```
@EnableCaching
```

```
public class ProductApplication {
```

```
    public static void main(String[] args) {
```

```
        SpringApplication.run(ProductApplication.class, args);
```

```
}  
}
```

2. Cache product data

```
@Service  
public class ProductService {  
    @Cacheable(value = "products", key = "#id")  
    public Product getProduct(Long id) {  
        System.out.println("Fetching from database...");  
        return productRepository.findById(id)  
            .orElseThrow(() ->  
                new RuntimeException("Product not found"));  
    }  
}
```

The important part is:

```
@Cacheable(value = "products", key = "#id")
```

It tells Spring:

"Before executing this method, check the cache."

First request

```
getProduct(101)
```

Cache doesn't contain product 101:

MISS

↓

Database

↓

Store in Redis

↓

Return product

Second request

getProduct(101)

Redis already contains it:

HIT



Return product

The method doesn't need to query the database again.

Cache Invalidation During Update

Suppose we update product 101. We can use:

```
@CacheEvict(value = "products", key = "#product.id")  
public Product updateProduct(Product product) {  
    return productRepository.save(product);  
}
```

This removes the old cached value.

Flow:

Update Product



Update Database



Remove old Redis value



Next GET request



Cache MISS



Database



Store latest data in Redis

This prevents users from receiving outdated cached information.

Step 8 — Implement Cache Invalidation

What is cache invalidation

Cache invalidation means removing or updating old data from Redis when the actual data changes in the database.

Example

Suppose Redis contains:

product:101

Name: Laptop

Price: ₹60,000

Database also contains:

Product 101 → ₹60,000

Now the product price changes.

Database

Product 101 → ₹55,000

But Redis may still have:

Redis

Product 101 → ₹60,000

This is **stale data**.

Therefore, after updating the database, we should invalidate or update Redis.

Option 1 — Invalidate/Delete cache

Update Database

↓

Delete product:101 from Redis

Then the next request will be:

Request

↓

Redis → MISS

↓

Database → ₹55,000



Store latest data in Redis



Response

Spring Boot example

```
@CacheEvict(value = "products", key = "#id")  
  
public void updateProduct(Long id, Product product) {  
    productRepository.save(product);  
}
```

@CacheEvict removes the old cached value.

Step 9 — Test Cache Hit / Miss

First Request — Cache Miss

Suppose we request:

GET /products/101

Redis doesn't have the product.

Client



Spring Boot



Redis



MISS



Database

The application gets the product from the database and stores it in Redis.

So:

First Request → Database → Redis

Second Request — Cache Hit

Now request the same product again:

GET /products/101

This time:

Client



Spring Boot



Redis



HIT



Response

The database is not queried.

So:

Second Request → Redis

Why is this important

It proves that Redis caching is working correctly.

You can test this using:

- Postman
- Browser
- cURL
- Swagger UI

Example using cURL

First request

curl http://localhost:8080/products/101

Expected application flow:

Redis → MISS

Database → Get product

Redis → Store product

Response

Second request

curl http://localhost:8080/products/101

Expected:

Redis → HIT

Response

How can we confirm it

Add a log inside the database method:

```
public Product getProduct(Long id) {  
    System.out.println("Fetching product from DATABASE");  
    return productRepository.findById(id)  
        .orElseThrow(() ->  
            new RuntimeException("Product not found"));  
}
```

First request:

Fetching product from DATABASE

Second request:

No database message

That indicates the second request was served from Redis.

Step 10 — Compare Performance

Without Cache

Every request goes to the database:

Request 1 → Database

Request 2 → Database

Request 3 → Database

Request 4 → Database

Request 5 → Database

For 1,000 requests:

1,000 requests



1,000 database queries

This can increase database load and response time.

With Cache

After the first request:

Request 1 → Database → Redis

Later requests:

Request 2 → Redis

Request 3 → Redis

Request 4 → Redis

Request 5 → Redis

For example:

1,000 requests



First request → Database

Remaining requests → Redis

This can significantly reduce database queries.

Measure: Without Cache vs With Cache

You should compare metrics such as:

1. Response time

Example:

Without Redis:

250 ms

With Redis:

30 ms

Lower response time is better.

2. Database queries

Example:

Without Cache:

1000 database queries

With Cache:

50 database queries

Fewer database queries means lower database load.

3. Throughput

How many requests the application can process in a given amount of time.

For example:

Without Cache → 100 requests/sec

With Cache → 500 requests/sec

Higher throughput is generally better.

4. Database CPU/load

Without caching:

Application



Many DB requests



High DB load

With caching:

Application



Redis handles many reads



Fewer DB requests



Lower DB load

Step 11 — Design Final Distributed Data Architecture

1. API

The **API** is the entry point for client requests.

For example:

GET /products/101

The request comes from:

Web Application

Mobile Application

Postman

Frontend



API

The API sends the request to the appropriate microservice.

2. Microservice

The **microservice** contains the business logic.

For example:

Product Service

It handles operations such as:

Get Product

Create Product

Update Product

Delete Product

The microservice communicates with both:

- Redis
- Database

3. Redis

Redis is used as the **cache**.

The microservice first checks Redis for frequently requested data.

Example:

GET /products/101



Product Service



Redis



Product found?

If found:

Redis → Product Service → API → Client

This is a **cache hit**.

If not found:

Redis → MISS



Database

This reduces the number of database queries.

4. Database

The database is the **persistent storage**.

For example:

MySQL / PostgreSQL

It stores the actual product information:

Product ID: 101

Name: Laptop

Price: ₹60,000

Stock: 20

The important concept is:

The database is the source of truth.

Redis is only a temporary copy of frequently accessed data.

5. Kafka

Database



Kafka



Other Services

Kafka is used for **event-driven communication** between services.

Example, suppose a product price changes:

Product Service



Database updated



ProductUpdated event



Kafka



Other Services

Other services can consume the event.

Example:

Product Service



Kafka



Order Service Search Service Notification Service

This allows services to communicate without being tightly coupled.

Example: Product Update

Suppose the user changes:

Laptop price:

₹60,000 → ₹55,000

The flow can be:

Client



API



Product Microservice



Update Database



Invalidate/Update Redis



Publish ProductUpdated event



Kafka



Other Services

This is a typical distributed architecture.

Component	Main Responsibility
API	Receive client requests
Microservice	Business logic
Redis	Fast temporary/cache storage
Database	Permanent/source-of-truth storage
Kafka	Asynchronous event communication
Other Services	Consume events and perform their own work

Think of it like this:

Redis → SPEED

Database → TRUTH

Kafka → COMMUNICATION

Microservice → BUSINESS LOGIC

API → ENTRY POINT

Step 12 — Document Consistency Considerations

1. Cache Stale Data

Stale data means Redis contains an old value while the database contains the latest value.

Example:

Database:

Price = ₹55,000

Redis:

Price = ₹60,000

Redis is now holding **stale data**.

If the application reads from Redis, the user may see:

₹60,000

even though the actual price is:

₹55,000

How can we reduce stale data

Use:

- TTL
- Cache invalidation
- Cache update
- Proper update strategy

2. Cache Invalidation

When the database changes, the related Redis entry should be removed or updated.

Example:

Old:

Database = ₹60,000

Redis = ₹60,000

Update:

Database = ₹55,000

Then:

Delete/Update Redis

Now:

Database = ₹55,000

Redis = removed/₹55,000

Spring Boot example

```
@CacheEvict(value = "products", key = "#id")  
  
public void updateProduct(Long id, Product product) {  
    productRepository.save(product);  
}
```

This removes the old cached value.

3. Database as Source of Truth

This is one of the most important concepts.

The **database should be considered the authoritative source of the data.**

Example:

Database:

Product 101

Price = ₹55,000

Redis:

Product 101

Price = ₹60,000

Which one should we trust?

Database.

Redis is a cache.

Therefore:

Database = Source of Truth

Redis = Cached Copy

If Redis fails:

Redis DOWN



Application



Database

The application should still be able to retrieve the data from the database.

4. Eventual Consistency

In distributed systems, different components may not become updated at exactly the same time.

This is called **eventual consistency**.

Example:

10:00:00

Database → ₹55,000

10:00:01

Redis → ₹60,000

10:00:02

Cache updated

10:00:03

Redis → ₹55,000

For a short period, the values were different.

Eventually they become consistent.

Database = ₹55,000

Redis = ₹55,000

That is **eventual consistency**.

5. Cache Failure Handling

What happens if Redis goes down

For example:

Client



Microservice



Redis ❌

The application should not necessarily stop working.

A good design can fall back to the database:

Request



Microservice



Redis



Redis unavailable



Database



Response

This is called a **cache fallback** or **cache failure handling** strategy.

Step 13 — Final Testing

1. Test Cache Hit

A **cache hit** occurs when the requested data already exists in Redis.

Example:

GET /products/101



Redis



FOUND



Return data

Test

First, load the product so it gets stored in Redis.

Then request the same product again.

Expected:

Redis → HIT → Response

The database should not be queried on the second request.

2. Test Cache Miss

A **cache miss** occurs when Redis doesn't contain the requested data.

Example:

GET /products/999



Redis



MISS



Database



Store in Redis



Response

Test

Request a product that isn't currently cached.

Expected:

Redis → MISS



Database



Redis



Response

This confirms your **cache-aside pattern** is working.

3. Test Cache Expiration

You need to verify that the **TTL** is working correctly.

Suppose you configure:

TTL = 60 seconds

Flow:

Product stored in Redis



Wait 60 seconds



Cache expires



Request product



Redis MISS



Database

Expected result

After the TTL expires, Redis should no longer return the old cached value.

4. Test Update

You need to verify that updating the database works correctly.

Example:

Initial:

Database:

Product 101 = ₹60,000

Redis:

Product 101 = ₹60,000

Update product:

₹60,000 → ₹55,000

Expected:

Update Database

↓

Invalidate/Update Redis

↓

Next GET

↓

Latest value = ₹55,000

The purpose is to make sure users don't receive outdated information.

5. Test Cache Invalidation

This is closely related to the previous test, but here you specifically verify that the old cache entry is removed.

For example:

Redis:

product:101 = ₹60,000

Update the product:

Database:

product:101 = ₹55,000

Then invalidate:

DELETE product:101 from Redis

Now:

Redis → MISS



Database → ₹55,000



Redis → Store latest value

Expected result

The old cached value should not be returned.

In Spring Boot, this is commonly done with:

```
@CacheEvict(value = "products", key = "#id")
```

6. Test Redis Unavailable

This is an important **failure-handling test**.

What happens if Redis is down?

For example:

Client



Spring Boot



Redis ❌

Your application should handle the failure gracefully.

Depending on your application's design, it can fall back to the database:

Request



Redis unavailable



Database



Return response

A Redis failure should not unnecessarily make the entire application unavailable when Redis is only being used as a cache.